

COMP 8967 — Summer 2026

Project Proposal

Retrieval-Augmented Generation (RAG) System for Question Answering

Team 10

Submitted To:

Dr. Prashanth Ranga
Supervisor

Academic Affiliate:

Dr. Shaon Shuvo

Submitted By:

Student Name	Student ID
Anurag Sharma	110171321
Prabhjot Singh	110195228
Harpreet Singh	110177411
Chetan Thakur	110213868

University of Windsor

Summer 2026 Term (June 1 – July 31, 2026)
June 2026

Table of Contents

1. Introduction	3
2. Motivation	4
3. Background Study	5
4. Project Goals	6
5. Project Roadmap	7
6. Resources	9
7. References	10

1. Introduction

Question Answering (QA) systems have become an integral part of how we interact with information in the digital age. From search engines to virtual assistants, users expect accurate, contextually relevant, and timely responses to their queries. Traditional QA systems, however, face significant limitations: they rely on static knowledge bases, struggle with domain-specific questions, and are prone to generating incorrect or hallucinated information when faced with unfamiliar queries.

Our project addresses these challenges by implementing a Retrieval-Augmented Generation (RAG) system. RAG represents a paradigm shift in how we approach question answering by combining the strengths of information retrieval with the generative capabilities of modern language models. Unlike traditional approaches that depend solely on parametric knowledge stored within a model, RAG systems dynamically retrieve relevant documents from an external knowledge base and use them to generate more accurate, grounded, and verifiable responses.

This project aims to build a complete, end-to-end RAG system capable of handling real-world question answering tasks using the MS MARCO dataset — a widely recognized benchmark for passage retrieval and QA. By integrating state-of-the-art embedding models, efficient vector indexing using FAISS, and carefully selected generation strategies, we seek to demonstrate how RAG can improve both the accuracy and reliability of QA systems.

2. Motivation

2.1 Why is this Problem Interesting?

The challenge of building effective QA systems sits at the intersection of several active research areas in artificial intelligence, including natural language processing, information retrieval, and representation learning. What makes this problem particularly interesting is the tension between two competing requirements: the need for systems that understand and reason about language, and the need for systems that can access and process vast amounts of factual information.

Pure generative models, while impressive in their ability to produce fluent text, suffer from fundamental limitations. They are trained on fixed corpora and cannot access information beyond what was present during training. More critically, they are prone to hallucination — generating plausible-sounding but factually incorrect information. This is particularly problematic in domains where accuracy is paramount, such as healthcare, legal advice, or educational applications.

2.2 Why is it Challenging?

Building an effective RAG system presents several key technical challenges:

1. **Retrieval Accuracy:** The system must identify the most relevant passages from potentially millions of documents. Poor retrieval directly leads to poor generation, even with sophisticated language models.
2. **Computational Efficiency:** Real-time querying requires fast similarity search over high-dimensional vector representations. Balancing accuracy with speed is a critical design consideration.
3. **Integration:** Effectively combining retrieved context with generative models requires careful prompt engineering and model selection.
4. **Evaluation:** Measuring the quality of generated answers is inherently more complex than evaluating retrieval alone, requiring both automated metrics and human judgment.

2.3 Who Benefits?

Several stakeholders stand to benefit from improved RAG systems:

- **End Users:** Anyone seeking information through search interfaces benefits from more accurate, comprehensive answers with source attribution.
- **Organizations:** Companies handling customer support, knowledge management, or internal documentation can deploy RAG systems to improve information access.
- **Researchers:** The RAG framework provides a testbed for exploring novel approaches to retrieval, representation learning, and generation.
- **Educational Platforms:** Students and educators can leverage RAG systems as tutoring aids that provide explanatory answers with citations.

3. Background Study

3.1 The RAG Framework

Retrieval-Augmented Generation was formally introduced by Lewis et al. (2020) [1] as a way to combine parametric and non-parametric memory in language models. The key insight is that rather than storing all knowledge in the model's parameters, we can maintain a separate, updatable knowledge base and retrieve relevant information at generation time. The original RAG architecture consists of two main components:

5. Retriever: A dense passage retrieval model that learns to encode documents and queries into a shared embedding space, allowing efficient similarity-based retrieval.
6. Generator: A sequence-to-sequence model (typically based on the Transformer architecture) that conditions its output on both the input query and the retrieved documents.

3.2 Embedding Models

Converting text into meaningful vector representations is fundamental to RAG systems. Two prominent approaches we will evaluate are:

BGE (BAAI General Embedding): A family of embedding models developed by the Beijing Academy of Artificial Intelligence [4]. The BGE-small variant offers a favorable trade-off between model size and performance, making it suitable for resource-constrained deployments.

E5 (Embeddings from Bidirectional Encoder Representations): A model family that emphasizes training with weak supervision signals, allowing it to learn robust representations across diverse domains [5]. E5-base provides a middle ground in terms of computational cost and embedding quality.

Both models leverage the SentenceTransformers framework [6], which provides a convenient interface for encoding text and can be fine-tuned for specific tasks.

3.3 Vector Similarity Search with FAISS

FAISS (Facebook AI Similarity Search) is a library developed by Meta Research for efficient similarity search and clustering of dense vectors [3]. In our RAG system, FAISS will serve as the vector database, indexing embeddings of all passages in our corpus. Key indexing strategies include:

- Flat Index: Provides exact nearest neighbor search by computing distances to all vectors. Suitable for smaller datasets.
- IVF (Inverted File Index): Partitions the vector space and only searches relevant partitions, trading slight accuracy loss for significant speed improvements.

- HNSW (Hierarchical Navigable Small World): Builds a graph structure enabling very fast approximate search with minimal accuracy degradation.

3.4 The MS MARCO Dataset

The Microsoft Machine Reading Comprehension (MS MARCO) dataset [2] is a large-scale real-world QA collection. Unlike synthetic datasets, MS MARCO contains real user queries from the Bing search engine, along with human-written answers and relevance judgments for passages. This realism makes it an ideal benchmark for evaluating RAG systems, as it reflects the variability and ambiguity of actual user queries. We will also supplement evaluation with SQuAD v2 for question answering benchmarking and a custom PDF document collection for domain-specific testing.

4. Project Goals

4.1 Overall Vision

Our project aims to develop and evaluate a complete Retrieval-Augmented Generation system for open-domain question answering. The system should demonstrate improved answer quality compared to baseline approaches while maintaining acceptable computational efficiency for real-time or near-real-time querying. The focus is on building a practical, well-evaluated pipeline — not on achieving state-of-the-art leaderboard performance.

4.2 Specific Objectives

7. Build a Functional RAG Pipeline: Implement all components necessary for end-to-end question answering, including document indexing, query processing, retrieval, and answer generation.
8. Evaluate Multiple Configurations: Compare different embedding models (BGE-small vs. E5-base) and indexing strategies (Flat vs. IVF vs. HNSW) to understand their trade-offs in terms of accuracy, latency, and resource usage.
9. Implement Hybrid Retrieval: Combine dense vector retrieval with BM25 sparse retrieval using score fusion techniques such as Reciprocal Rank Fusion (RRF).
10. Benchmark on MS MARCO and SQuAD v2: Achieve competitive performance on standard evaluation metrics including Mean Reciprocal Rank (MRR), Recall@K, Precision@K, and Faithfulness.
11. Document and Replicate: Create comprehensive documentation and reproducible experiments, enabling others to understand and build upon our work.
12. Deliver a Usable Prototype: Provide a demonstration interface showing the system's capabilities on sample queries.

4.3 What We Are NOT Committing To

To ensure project feasibility within the eight-week timeframe, we explicitly exclude the following:

- Training new embedding models from scratch (we will use pre-trained models)
- Achieving state-of-the-art performance on MS MARCO leaderboards (our focus is on understanding, not competition)
- Multi-modal retrieval (handling images, videos, or structured data)
- Real-time deployment at web scale (we target prototype-scale deployment)
- Fine-tuning large language models for generation (we will use pre-trained or API-based models)

5. Project Roadmap

The project spans from June 1, 2026 to July 31, 2026 (9 weeks). Each team member is assigned dedicated tasks every week to ensure equal, individual contributions throughout the project. The work distribution is designed so that all four members carry equivalent load and maintain distinct, assessable deliverables.

Week 1 (Jun 1–7): Foundation & Proposal

Team Member	Task	Deliverable
Anurag Sharma	Write project proposal — Introduction and Problem Statement sections	Proposal draft (Intro)
Prabhjot Singh	Literature review — RAG framework and retrieval papers (Lewis et al., DPR)	Lit review summary (retrieval)
Harpreet Singh	Set up GitHub repository, folder structure, README, and branching conventions	Git repo with project structure
Chetan Thakur	Literature review — Embedding models (BGE, E5, SentenceTransformers)	Lit review summary (embeddings)

Week 2 (Jun 8–14): Environment Setup & Dataset Download

Team Member	Task	Deliverable
Anurag Sharma	Install and configure Python environment: PyTorch, Transformers, FAISS, LangChain	Working dev environment
Prabhjot Singh	Set up Jupyter notebooks, pytest, Black/Ruff linting, and project templates	Testing & notebook framework
Harpreet Singh	Download, verify, and document MS MARCO dataset format and splits	MS MARCO dataset ready
Chetan Thakur	Download, verify, and document SQuAD v2 dataset and explore schema	SQuAD v2 dataset ready

Week 3 (Jun 15–21): Data Pipeline

Team Member	Task	Deliverable
Anurag Sharma	Build MS MARCO data loading and cleaning script; parse passages, queries, and relevance labels	Data loading pipeline
Prabhjot Singh	Implement text chunking pipeline (fixed-size and sliding window strategies)	Chunking module
Harpreet Singh	Build data preprocessing and normalization pipeline (lowercasing, stop-word handling, deduplication)	Preprocessing pipeline

Team Member	Task	Deliverable
Chetan Thakur	Prepare custom PDF domain-specific dataset: extraction, chunking, and formatting	Custom PDF dataset ready

Week 4 (Jun 22–28): Baseline Retrieval

Team Member	Task	Deliverable
Anurag Sharma	Implement BM25 baseline retrieval using rank-bm25 library with query scoring	BM25 retrieval module
Prabhjot Singh	Implement TF-IDF baseline retrieval using scikit-learn vectorizer and cosine similarity	TF-IDF retrieval module
Harpreet Singh	Build evaluation framework: Recall@K, Precision@K, and MRR scoring scripts	Evaluation framework
Chetan Thakur	Document and visualize baseline retrieval results (tables and graphs for BM25 and TF-IDF)	Baseline results report

Week 5 (Jun 29–Jul 5): Embedding Models & FAISS Indexing

Team Member	Task	Deliverable
Anurag Sharma	Integrate BGE-small embedding model; encode passages and build FAISS Flat index	BGE-small + FAISS Flat index
Prabhjot Singh	Integrate E5-base embedding model; encode passages and build FAISS Flat index	E5-base + FAISS Flat index
Harpreet Singh	Implement FAISS IVF (Inverted File Index) with nprobe tuning for scalable search	FAISS IVF index
Chetan Thakur	Implement FAISS HNSW (Hierarchical Navigable Small World) index with M and ef tuning	FAISS HNSW index

Week 6 (Jul 6–12): Hybrid Retrieval & Optimization

Team Member	Task	Deliverable
Anurag Sharma	Implement hybrid retrieval score fusion (BM25 + dense scores using Reciprocal Rank Fusion)	Hybrid retrieval module
Prabhjot Singh	Implement query expansion using LLM to expand queries before the retrieval pass	Query expansion module
Harpreet Singh	Conduct embedding comparison analysis: BGE vs E5 on MRR, latency, and memory usage	Embedding comparison report
Chetan Thakur	Run chunk size and overlap experiments (128/256/512 tokens) and analyze impact on retrieval quality	Chunking analysis report

Week 7 (Jul 13–19): LLM Integration & Advanced Optimization

Team Member	Task	Deliverable
Anurag Sharma	Integrate LLM API (OpenAI / LLaMA) and connect retriever output to the generator	End-to-end RAG pipeline
Prabhjot Singh	Perform prompt engineering and optimization: context formatting and few-shot templates	Prompt templates library
Harpreet Singh	Integrate cross-encoder reranking model to reorder retrieved passages before generation	Reranking module
Chetan Thakur	Build context compression module to filter irrelevant retrieved passages before generation	Context compression module

Week 8 (Jul 20–26): Full Evaluation & UI Prototype

Team Member	Task	Deliverable
Anurag Sharma	Run full MS MARCO validation evaluation: MRR@10 and Recall@K on the validation set	MS MARCO evaluation results
Prabhjot Singh	Run generation quality evaluation: Faithfulness, BLEU, and ROUGE metric scoring	Generation quality report
Harpreet Singh	Conduct speed and accuracy benchmarking: latency per index type and query throughput	Benchmarking report
Chetan Thakur	Build simple UI prototype for end-to-end demonstration on sample queries	Working UI demo

Week 9 (Jul 27–31): Final Report & Presentation

Team Member	Task	Deliverable
Anurag Sharma	Write IEEE report sections: Results and Discussion; include tables, graphs, and failure case analysis	IEEE Results & Discussion
Prabhjot Singh	Write IEEE report sections: Methodology and Experimental Setup; describe system architecture and pipeline	IEEE Methodology section
Harpreet Singh	Write IEEE report sections: Related Work and Introduction; cite and position against prior literature	IEEE Related Work section
Chetan Thakur	Prepare final presentation slides with speaker notes, demo walkthrough, and conclusion summary	Final presentation deck

5.1 Individual Role Summary

The table below summarizes the primary area of ownership for each team member across the full project:

Team Member	Primary Role	Key Contributions
Anurag Sharma	Evaluation & Retrieval Lead	BM25, hybrid fusion, LLM integration, MS MARCO eval, IEEE Results
Prabhjot Singh	Data & Generation Lead	TF-IDF, chunking, E5-base, query expansion, generation eval, IEEE Methodology
Harpreet Singh	Infrastructure & Indexing Lead	Repo setup, FAISS IVF, evaluation framework, reranking, benchmarking, IEEE Related Work
Chetan Thakur	Optimization & Presentation Lead	FAISS HNSW, chunk experiments, context compression, UI prototype, final slides

6. Resources

6.1 Required Skills

Team members will develop or apply expertise in the following areas:

- Python Programming: Core implementation language; familiarity with NumPy, Pandas, PyTorch/TensorFlow
- Natural Language Processing: Understanding of text processing, tokenization, and language model usage
- Machine Learning Fundamentals: Embeddings, similarity metrics, and evaluation metrics
- Version Control: Git for collaborative development and code management
- Technical Writing: Documenting experiments, writing reports, and preparing presentations

6.2 Technology Stack

Component	Technology / Tool
Programming Language	Python 3.9+
Core Libraries	NumPy, Pandas, PyTorch / TensorFlow
Embedding Models	BGE-small, E5-base, SentenceTransformers, HuggingFace Transformers
Vector Database	FAISS (Flat, IVF, HNSW indexes)
RAG Framework	LangChain or LlamaIndex (optional orchestration)
Retrieval Method	Dense retrieval + BM25 hybrid (Reciprocal Rank Fusion)
LLM / Generation	OpenAI API or open-source LLM (LLaMA, Mistral) via API or local inference
Datasets	MS MARCO, SQuAD v2, Custom PDF collection
Evaluation Metrics	Recall@K, Precision@K, MRR, Faithfulness, BLEU, ROUGE
Development Tools	Jupyter Notebook, Git / GitHub, pytest, Black / Ruff
Visualization	Matplotlib, Seaborn

6.3 Hardware Requirements

- CPU: Modern multi-core processor for efficient data processing
- RAM: Minimum 16 GB; 32 GB+ recommended for large-scale indexing operations
- GPU: CUDA-compatible GPU beneficial for embedding generation; not strictly required if using APIs
- Storage: 50 GB+ for dataset storage, model checkpoints, and FAISS indexes

6.4 Project Management Tools

- GitHub Projects: For task tracking, sprint planning, and pull request reviews
- Shared Document Repository: Google Drive for collaborative writing and file sharing
- Communication: Group chat / meeting platform for regular weekly coordination
- Issue Tracking: GitHub Issues for bug reports and feature tracking

7. References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [2] Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwari, M., Majumder, R., ... & Goodman, L. (2016). MS MARCO: A Human Generated Machine Reading Comprehension Dataset. *arXiv preprint arXiv:1611.08231*.
- [3] Johnson, J., Douze, M., & Jegou, H. (2019). Billion-scale similarity search with GPUs. *arXiv preprint arXiv:1902.09129*.
- [4] Xiao, S., Liu, Z., Zhang, P., Pei, Y., & Shi, Y. (2023). BGE-M3: Multi-Functionality, Multi-Linguality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *Proceedings of EMNLP*, 12098–12119.
- [5] Wang, L., Yang, N., Wei, F., Chang, B., Yang, M., Li, J., ... & Zhang, Y. (2024). Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03527*.
- [6] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP*, 3982–3992.
- [7] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of EMNLP*, 6690–6700.
- [8] Zhao, W. X., et al. (2023). A Survey of Large Language Models. *arXiv preprint arXiv:2303.18223*.
- [9] Gao, T., Yao, S., & Chen, D. (2021). SimCSE: Simple Contrastive Learning of Sentence Embeddings. *Proceedings of EMNLP*, 6894–6910.
- [10] Guo, J., Fiscus, J., & Guo, Y. (2023). MS MARCO: A Comprehensive Overview and Analysis of the MS MARCO Dataset. *arXiv preprint arXiv:2310.10983*.